Exercise on Cost-effective Regression Testing
Week 5: Regression Testing

Learning Goal:

- Understand and apply different test prioritization and minimization, such as greedy-based minimisation
- Apply and implement diversity-based testing to prioritise system-level tests.

There are two options to complete this exercise. You can choose to implement your own test prioritisation tool using the diversity-based testing algorithms (Option 1), or do the calculations on a smaller example manually (Option 2).

Option 1: Automated Test Optimisation

Example Case: Mozilla Test Prioritiser

You were hired as a Test Engineer at Mozilla. They have a test suite with more than 200 manual test cases (see files) and you were asked to implement a tool for diversity-based test (DBT) prioritisation. From the DBT steps, the tests are already encoded into a file (HTML or Text). You can find the tests in **Canvas > Files > Other material > Exercise Files > mozilla_tcs.zip**

The goal is to output: (i) a distance matrix with all the distance values between those pairs, and (ii) a file containing the prioritised test suite. For (ii) we recommend outputting CSV files, where the columns are: Rank, TestID, TestDescription. The Rank is the order in which the test will be executed. The test ID or description can be extracted from the test artefacts (e.g., filename, the first sentence in the test, etc.)

Write a program that will perform the test prioritisation. We recommend finding existing libraries for calculating string distances. We recommend using Jaccard as it is faster (Levenshtein or the edit-distance is a slower but feasible alternative).

Here are some recommendations of libraries:

Python:

- https://www.nltk.org/#natural-language-toolkit

Java:

- Apache Commons Lang 3: https://commons.apache.org/proper/commons-lang/
    - StringUtils class
- https://github.com/tdebatty/java-string-similarity

Option 2: Manual Test Optimisation

Example Case: MyMovieApp Application

Here we will use as a case a hypothetical set of test cases for a web application to register and retrieve data about movies and their cast members (actors and actresses). The app supports the addition of movie and cast entries manually or automatically by providing a URL to the corresponding item on IMDB (International Movie Database).

For the sake of simplicity we chose **10 test cases** exercising distinct, yet similar scenarios of the application. Each test case has a title (ID), a set of preconditions and postconditions, and finally a sequence of steps. The test cases also use data that can be stored in a file and automatically replaced when building a test plan. The data used in test cases are marked between angled brackets "<< >>".

You are not expected to change, create or analyse details of the test cases. Rather, you will use them, as they are, to execute the techniques. Here we want to select, minimise and classify the set of test cases to reduce costs in testing. **The test artefacts can be found in the appendix to this exercise.**

Part 1 - Test Case Selection and Prioritization

We begin with the Diversity-based test selection (DBT). Use the JaccardIndex to analyse the text diversity of steps, preconditions and postconditions of the tests all together. For simplicity, implement a script that parses the text information and calculates the JaccardIndex as words (alternatively, you can find and use string libraries that calculate Jaccard index using the actual algorithm based on q-grams)

Q1.1 - **Calculate all similarity values** and fill in the similarity matrix below. We suggest a precision of two digits of precision for the similarity values (e.g.: 0.78).

Remember: You can ignore the diagonal and focus only on the upper diagonal of the matrix, since the similarity function is symmetrical.

Similarity Matrix :

	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10
TC1	-	0.52	0.24	0.22	0.22	0.22	0.24	0.2	0.26	0.38
TC2	-	-								
TC3	-	-	-							
TC4	-	-	-	-						
TC5	-	-	-	-	-					
TC6	-	-	-	-	-	-				
TC7	-	-	-	-	-	-	-			
TC8	-	-	-	-	-	-	-	-		
TC9	-	-	-	-	-	-	-	-	-	
TC10	-	-	-	-	-	-	-	-	-	-

Q1.2 – Based on the matrix, prioritise the test suite by executing first the tests that are more diverse to the others. Then provide **3 subsets of test cases,** covering the top 20%, 40%, 60% of the most diverse tests. For instance, you can write only the ID of the selected test cases Ex: T_20 = {TC1, TC8}. Use two methods for identifying the subsets: i) Similarity-based test case prioritization and ii) Distance-based test case prioritization (Max-min method).

Selected subsets:

T_60 =
T_40 =
T_20 =

Q1.3 - How do we compare which prioritised test suite is better? After executing and debugging the 10 test cases, we find the traceability below between test cases and faults (not failures!). Reflect on how many faults are revealed by the different subsets (T_60, T_40, T_20). You can choose which subsets to use T_60, T_40, T_20 from Q1.2 similarity based method or T_60, T_40, T_20 from Q1.2 distance based method.

Q1.4 - A colleague suggested a different prioritised test suite: **P-Sam: {TC8, TC6, TC2, TC3, TC4, TC7, TC1, TC5, TC10, TC9}.** Which prioritised test suite is better at revealing faults earlier (APFD)? Justify why a test suite is better than the other. Compare three prioritized test suites: Test suite prioritised by similarity method **P-Similarity: {TC1, TC2, TC4, TC8, TC9, TC10, TC5, TC7, TC6, TC3}**, Test suite prioritised by distance (max-min method) **P-Distance: {TC8, TC2, TC10, TC1, TC4, TC9, TC5, TC7, TC6, TC3}** and P-Sam test suite.

	F1	F2	F3	F4	F5	F6
TC1	x		x			
TC2						
TC3		x				x
TC4						
TC5	x	x				x
TC6						
TC7			x			
TC8					x	
TC9				x		x
TC10	x					x

Q1.5 – Explain how different selection percentages (T_20, T_40, T_60) for the prioritized test suite impact the coverage of the scenarios represented by the 10 test cases. For example, assume each test case takes approximately 5 minutes to execute. How much time is required to run each test suite? How many faults does each suite detect? Discuss the trade-off between test suite execution time and the number of faults detected.

Part 2 - Test Suite Minimisation (Optional)

Traceability between the different elements of the app specification (features and data) and test cases.

	Login	Create	Delete	Search	Movie	Cast	IMDB	Manual
TC1	x							
TC2	x							
TC3		x			x			x
TC4		x			x		x	

	Login	Create	Delete	Search	Movie	Cast	IMDB	Manual
TC1	x							
TC2	x							
TC5		x				x		x
TC6		x				x	x	
TC7			x		x			x
TC8			x		x	x		x
TC9				x	x			
TC10	x							

The customer explained to us how she wanted the features of the web app. The **traceability** above is provided between the 10 test cases (rows), and the requirements (columns).

Q2.1 - Provide a minimised test suite **T_min** that covers all requirements.

Q2.2 – How does your minimised test suite compare to the three subsets from question 1.2 above?

Appendix: Exercise "Option 2 – Manual Test Optimization"

Jaccard Index: This is a similarity score for how much "overlap" in tokens two strings have. To obtain the distance, you need to do: `JaccDist(TCi, TCj) = 1 - J(TCi,TCj)`

$$J(TC_i, TC_j) = \frac{|TC_i \cap TC_j|}{|TC_i \cup TC_j|}$$

tc8	tc2	tc10	TC1	tc4	TC9	tc5	TC7	tc6	tc3

1. Manual Test Cases

TC1: Valid Login
Pre-condition:
User <<valid_user>> is registered App login screen is shown
Steps
Access the MyMovieApp
In the login field, type <<valid_user.email>>
In the password field, type << valid_user.pwd>>
Press the "Login" button
Post-condition:
App home screen is shown

TC2: Invalid Login
Pre-condition:
User <<invalid_user>> is not registered
Steps
Access the MyMovieApp
In the login field, type <<invalid_user.mail>>
In the Password field, type <<invalid_user.pwd>>
Press the "Login" button
Post-condition:
Message “User not registered” is shown

TC3: Create movie entry (manual)
Pre-condition:
<<user>> is logged in successfully <<movie>> was not created yet in <<user>>’s profile App home screen is shown
Steps
Click the "Add movie" button
Choose the "manual entry" feature
In the title field type, <<movie>>
Press the " Add" button
Post-condition:
Message "Movie added successfully" is shown Updated table of movies is shown in main screen

TC4: Create movie entry (IMDB)
Pre-condition:
<<user>> is logged in successfully <<movie>> was not created yet in <<user>>’s profile App home screen is shown <<movie_url>> is accessible via Internet
Steps
Click the "Add movie" button
Choose the "Import from IMDB" feature
In the title field type, <<movie_url>>
Press the " Add" button
Post-condition:
Message "Movie added successfully" is shown Updated table of movies is shown in main screen

TC5: Create cast member entry (manual)
Pre-condition:
<<user>> is logged in successfully <<cast>> was not created yet in <<user>>’s profile App home screen is shown
Steps
Click the "Add movie actor/actress" button
Choose the "manual entry" feature
In the Name field type, <<cast.name>>
In the Movie Title field type, <<cast.movies>>
Press the "Add" button
Post-condition:
Updated table of cast members is shown in main screen Message "Actor/actress added successfully" is shown

TC6: Create cast member entry (IMDB)
Pre-condition:
<<user>> is logged in successfully <<cast>> was not created yet in <<user>>’s profile App home screen is shown
Steps
Click the "Add movie actor/actress" button
Choose the "Import from IMDB" feature

In the URL field type, <<cast.url>>
Press the " Add" button
Post-condition:
Updated table of cast members is shown in main screen Message "Actor/actress added successfully" is shown

TC7: Delete a movie entry (by title)
Pre-condition:
<<user>> is logged in successfully <<movie>> was created in <<user>>'s profile App home screen is shown
Steps
Click the "Delete a movie" button
Choose the "By title" feature
In the title field type, <<movie.title>>
Press the " Delete" button
Post-condition:
Updated table of movies is shown in main screen Dialog shows the "Movie deleted successfully" message

TC8: Delete a movie entry (by cast)
Pre-condition:
<<user>> is logged in successfully At least one title from << cast.movies>> was created in <<user>>'s profile <<cast>> was created in <<user>>'s profile App home screen is shown
Steps
Click the "Delete a movie" button
Choose the "By cast member" feature
In the actor/actress field type, <<cast.name>>
Press the "Delete" button
Post-condition:
Updated table of movies is shown in main screen Dialog shows the "Movie deleted successfully" message

TC9: Search movie entries (by year)
Pre-condition:

<<user>> is logged in successfully <<movies>> was created in <<user>>'s profile App home screen is shown
Steps
Click the "Search a movie" button
In the Year field type, <<movie.year>>
Press the "Search" button
Post-condition:
Updated table of movies is shown in main screen Dialog shows the "Movie deleted successfully" message

TC10: Logout
Pre-condition:
<<user>> is logged in successfully App home screen is shown
Steps
Click the "Logout" button
Press the "Confirm" button
Post-condition:
App login screen is shown

2. Set of words in each test case.

TC1	[<<valid_user.email>>, <<valid_user.pwd>>, <<valid_user>>, access, app, button, field, home, in, is, login, mymovieapp, password, press, registered, screen, shown, the, type, user]
TC2	[<<invalid_user.mail>>, <<invalid_user.pwd>>, <<invalid_user>>, access, button, field, in, is, login, message, mymovieapp, not, password, press, registered, registered", shown, the, type, user, "user]
TC3	[<<movie>>, <<user>>, <<user>>'s, add, added, app, button, choose, click, created, entry, feature, field, home, in, is, logged, main, manual, message, movie, movies, not, of, press, profile, screen, shown, successfully, table, the, title, type, updated, was, yet]
TC4	[<<movie>>, <<movie_url>>, <<user>>, <<user>>'s, accessible, add, added, app, button, choose, click, created, feature, field, from, home, imdb, import, in, internet, is, logged, main, message, movie, movies, not, of, press, profile, screen, shown, successfully, table, the, title, type, updated, via, was, yet]

TC5	[<<cast.movies>>, <<cast.name>>, <<cast>>, <<user>>, <<user>>'s, actor/actress, add, added, app, button, cast, choose, click, created, entry, feature, field, home, in, is, logged, main, manual, members, message, movie, name, not, of, press, profile, screen, shown, successfully, table, the, title, type, updated, was, yet]
TC6	[<<cast.url>>, <<cast>>, <<user>>, <<user>>'s, actor/actress, add, added, app, button, cast, choose, click, created, feature, field, from, home, imdb, import, in, is, logged, main, members, message, movie, not, of, press, profile, screen, shown, successfully, table, the, type, updated, url, was, yet]
TC7	[<<movie.title>>, <<movie>>, <<user>>, <<user>>'s, a, app, button, by, choose, click, created, delete, deleted, dialog, feature, field, home, in, is, logged, main, message, movie, movies, of, press, profile, screen, shown, shows, successfully, table, the, title, type, updated, was]
TC8	[<<cast.movies>>, <<cast.name>>, <<cast>>, <<user>>, <<user>>'s, a, actor/actress, app, at, button, by, cast, choose, click, created, delete, deleted, dialog, feature, field, from, home, in, is, least, logged, main, member, message, movie, movies, of, one, press, profile, screen, shown, shows, successfully, table, the, title, type, updated, was]
TC9	[<<movie.year>>, <<movies>>, <<user>>, <<user>>'s, a, app, button, click, created, deleted, dialog, field, home, in, is, logged, main, message, movie, movies, of, press, profile, screen, search, shown, shows, successfully, table, the, type, updated, was, year]
TC10	[<<user>>, app, button, click, confirm, home, in, is, logged, login, logout, press, screen, shown, successfully, the]

3. Sizes of the test cases				
|TC1| = 20	|TC2| = 21	|TC3| = 36	|TC4| = 41	|TC5| = 41
|TC6| = 40	|TC7| = 37	|TC8| = 45	|TC9| = 34	|TC10| = 16

4. Values for <u>intersections</u> between all pairs			
|TC1 & TC2|= 14	|TC2 & TC6|= 10	|TC4 & TC5|= 32	|TC6 & TC8|= 31
|TC1 & TC3|= 11	|TC2 & TC7|= 9	|TC4 & TC6|= 34	|TC6 & TC9|= 25
|TC1 & TC4|= 11	|TC2 & TC8|= 9	|TC4 & TC7|= 30	|TC6 & TC10|= 13
|TC1 & TC5|= 11	|TC2 & TC9|= 9	|TC4 & TC8|= 30	|TC7 & TC8|= 35
|TC1 & TC6|= 11	|TC2 & TC10|= 7	|TC4 & TC9|= 26	|TC7 & TC9|= 30
|TC1 & TC7|= 11	|TC3 & TC4|= 34	|TC4 & TC10|= 13	|TC7 & TC10|= 13
|TC1 & TC8|= 11	|TC3 & TC5|= 34	|TC5 & TC6|= 35	|TC8 & TC9|= 30
|TC1 & TC9|= 11	|TC3 & TC6|= 31	|TC5 & TC7|= 28	|TC8 & TC10|= 13

|TC1 & TC10|= 10	|TC3 & TC7|= 30	|TC5 & TC8|= 33	|TC9 & TC10|= 13
|TC2 & TC3|= 10	|TC3 & TC8|= 29	|TC5 & TC9|= 25	
|TC2 & TC4|= 10	|TC3 & TC9|= 26	|TC5 & TC10|= 13	
|TC2 & TC5|= 10	|TC3 & TC10|= 13	|TC6 & TC7|= 27	

5. Values for the <u>unions</u> between all pairs of test cases			
|TC1 U TC2 | = 27	|TC2 U TC6 | = 51	|TC4 U TC5 | = 50	|TC6 U TC8 | = 54
|TC1 U TC3 | = 45	|TC2 U TC7 | = 49	|TC4 U TC6 | = 47	|TC6 U TC9 | = 49
|TC1 U TC4 | = 50	|TC2 U TC8 | = 57	|TC4 U TC7 | = 48	|TC6 U TC10 | = 43
|TC1 U TC5 | = 50	|TC2 U TC9 | = 46	|TC4 U TC8 | = 56	|TC7 U TC8 | = 47
|TC1 U TC6 | = 49	|TC2 U TC10 | = 30	|TC4 U TC9 | = 49	|TC7 U TC9 | = 41
|TC1 U TC7 | = 46	|TC3 U TC4 | = 43	|TC4 U TC10 | = 44	|TC7 U TC10 | = 40
|TC1 U TC8 | = 54	|TC3 U TC5 | = 43	|TC5 U TC6 | = 46	|TC8 U TC9 | = 49
|TC1 U TC9 | = 43	|TC3 U TC6 | = 45	|TC5 U TC7 | = 50	|TC8 U TC10 | = 48
|TC1 U TC10 | = 26	|TC3 U TC7 | = 43	|TC5 U TC8 | = 53	|TC9 U TC10 | = 37
|TC2 U TC3 | = 47	|TC3 U TC8 | = 52	|TC5 U TC9 | = 50	
|TC2 U TC4 | = 52	|TC3 U TC9 | = 44	|TC5 U TC10 | = 44	
|TC2 U TC5 | = 52	|TC3 U TC10 | = 39	|TC6 U TC7 | = 50	